

Introduction to Formal Grammar

What is Grammar?

- **Definition:** The grammar of a language defines its alphabet and lexicon.
- **Syntax:** Specifies valid sequences of symbols forming phrases and sentences.
- **Production Rules:** A grammar consists of rules to transform strings.

Grammar of a Language

- The grammar of a language defines its alphabet and lexicon. Through syntax, it specifies the valid sequences of symbols that form well-formed phrases and sentences.
- A grammar consists of a set of production rules for transforming strings.
- To generate a string in the language, you start with a string consisting of a single start symbol.

Grammar of a Language

- Production rules are then applied, in any order, until the string contains neither the start symbol nor any designated nonterminal symbols.
- The language formed by the grammar consists of all the distinct strings that can be generated this way.

Grammar of a Language

- A particular sequence of production rules applied to the start symbol results in a unique string in the language.
- If there are multiple ways to generate the same string, the grammar is considered ambiguous.

A grammar

- A grammar is often viewed as a language generator.
- However, it can also serve as the basis for a "**recognizer**". A function in computing that determines whether a given string belongs to the language or is grammatically incorrect.
- To describe such “recognizers”, formal language theory utilizes automata theory.
- An important result of automata theory is that it is impossible to design a recognizer for certain formal languages.

String Generation Process

- Start with a **single start symbol**.
- Apply **production rules** until the string contains no start or nonterminal symbols.
- **Language**: Composed of all distinct strings generated.
- If multiple ways to generate the same string → **Ambiguity**.

Grammar as Generator and Recognizer

- **Generator:** Produces strings in the language.
- **Recognizer:** Checks if a string belongs to the language.
- **Automata Theory:** Describes recognizers, showing that some languages cannot have a recognizer.

The Syntax of Grammars

In the classic formalization of generative grammar, a grammar G consists of the following components:

- A finite set N of non-terminal symbols, none of which appear in strings formed by G .
- A finite set Σ (Sigma) of terminal symbols, disjoint from N .

The Syntax of Grammars

- A finite set P of production rules, each of the form $X \rightarrow YX$ or $X \rightarrow Y$, where X is a string of symbols (the "head"), and Y is another string of symbols (the "body").
 - If the body is empty (i.e., the rule represents an empty string), it may be denoted using special symbols such as ϵ (epsilon) or $'_'$.
- A distinguished start symbol S .

The Syntax of Grammars

Thus, a grammar is formally defined as the tuple $G = (N, \Sigma, S, P)$

where:

- N is the set of non-terminal symbols.
- Σ (Sigma) is the set of terminal symbols (the alphabet).
- S is the start symbol.
- P is the set of production rules.

Terminals (T)

- These are the basic symbols or tokens that cannot be broken down further.
- In natural language, terminals are words or punctuation marks.
- In programming languages, terminals are keywords, operators, or identifiers.
- Example: In the sentence "The cat sleeps," the terminals are "The," "cat," and "sleeps."

Non-terminals (N)

- These are variables that represent groups of symbols or patterns.
- Non-terminals are defined using production rules.

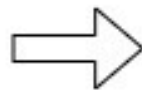
Production Rules (P)

- These are rules that define how sequences of terminals and non-terminals can replace non-terminals.
- Each rule has a left-hand side (a non-terminal) and a right-hand side (a sequence of terminals and/or non-terminals).
- Example: $\langle \text{sentence} \rangle \rightarrow \langle \text{subject} \rangle \langle \text{verb} \rangle \langle \text{object} \rangle$.

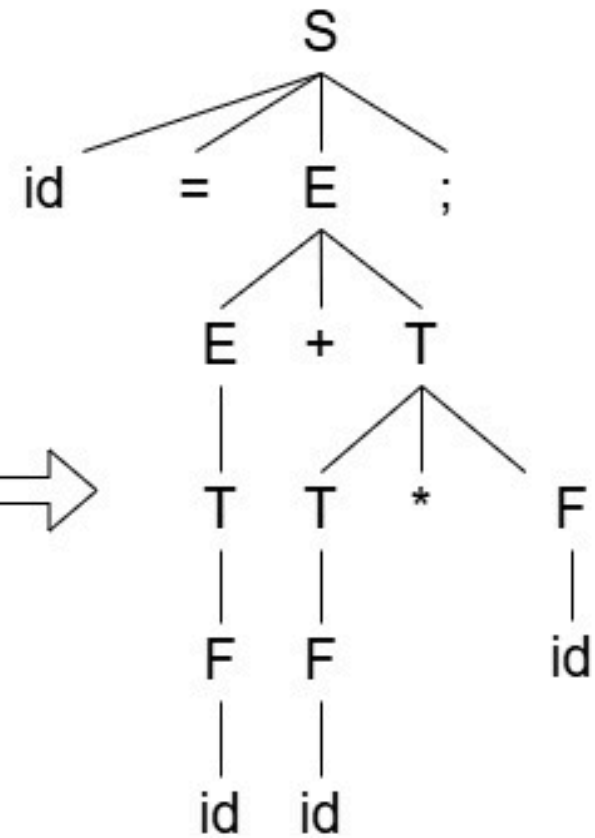
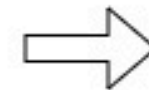
Start Symbol (S)

- This is a special non-terminal that represents the initial symbol from which all valid strings in the language are derived.
- Example: In a grammar for sentences, the start symbol might be <sentence>

Lexemes	Tokens
x	identifier
=	operator
a	identifier
+	operator
b	identifier
*	operator
c	identifier



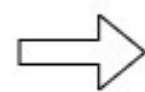
**Syntax
Analysis**



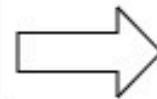
Parse Tree

Lexemes	Tokens
x	identifier
=	operator
a	identifier
+	operator
b	identifier
*	operator
c	identifier

x = a + b*c;



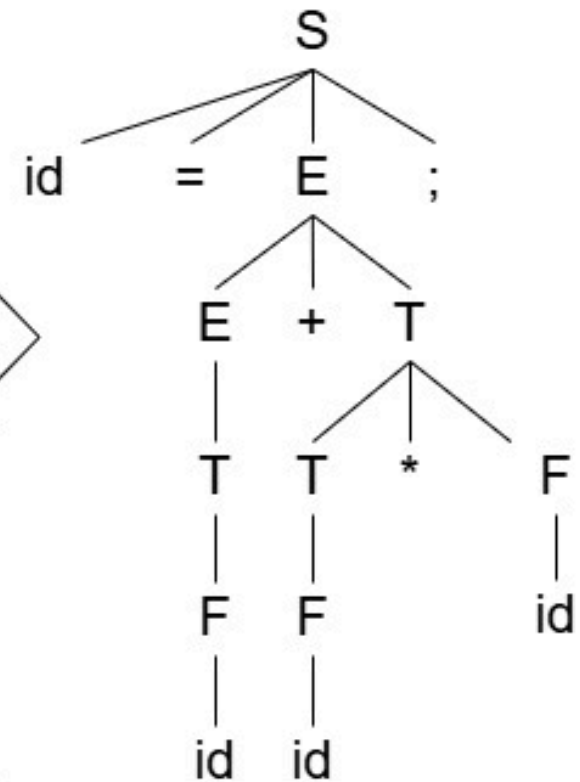
**Syntax
Analysis**



$S \rightarrow id = E ;$
 $E \rightarrow E + T | T$
 $T \rightarrow T * F | F$
 $F \rightarrow id$

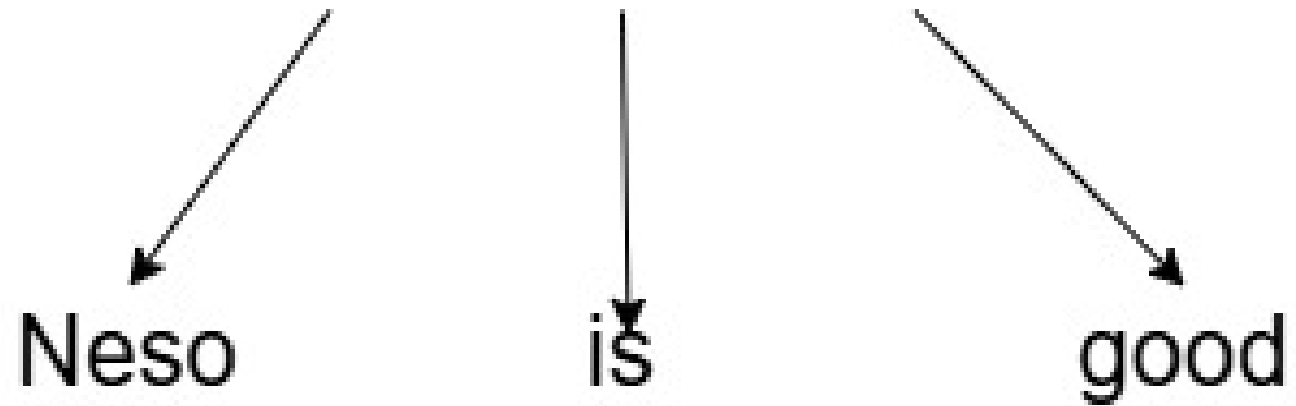
**Context-Free
Grammar (CFG)**

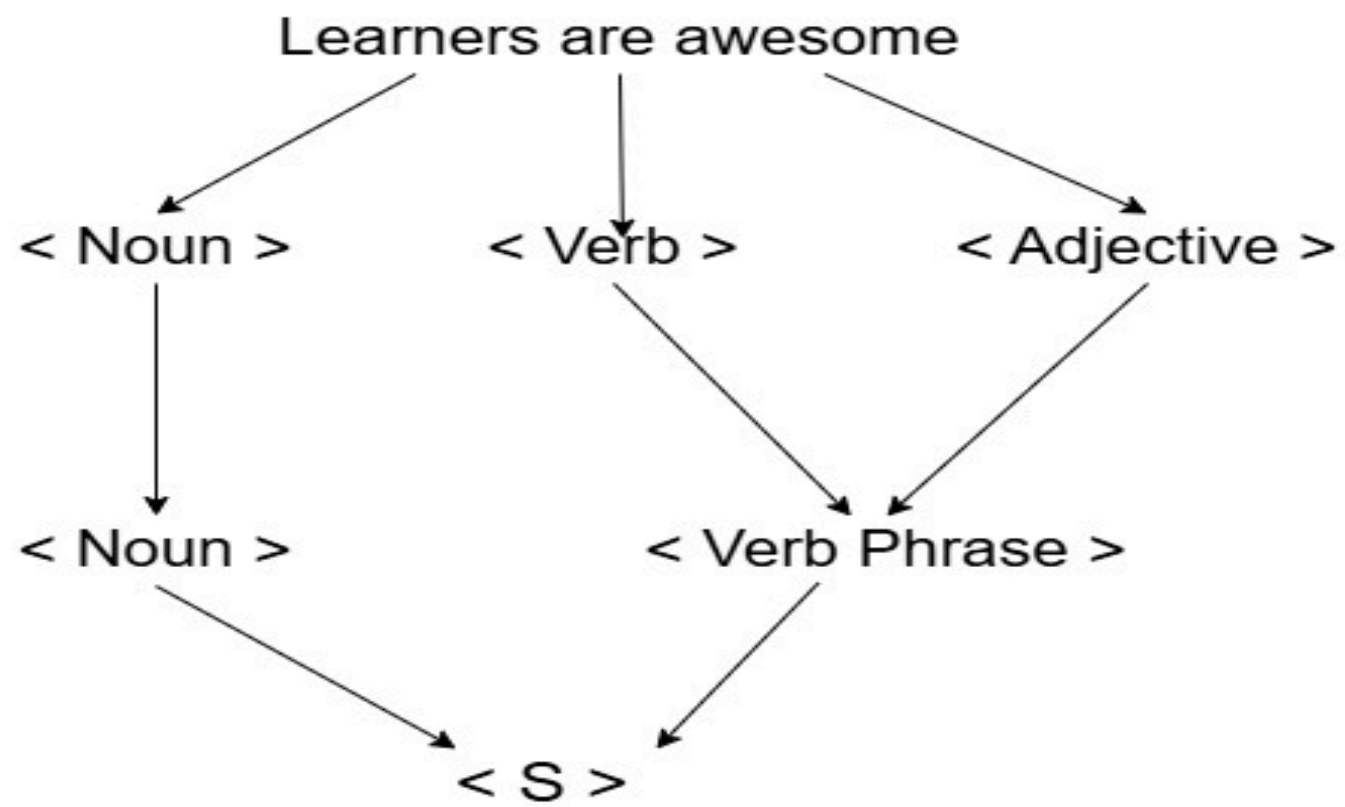
id = id + id * id ;

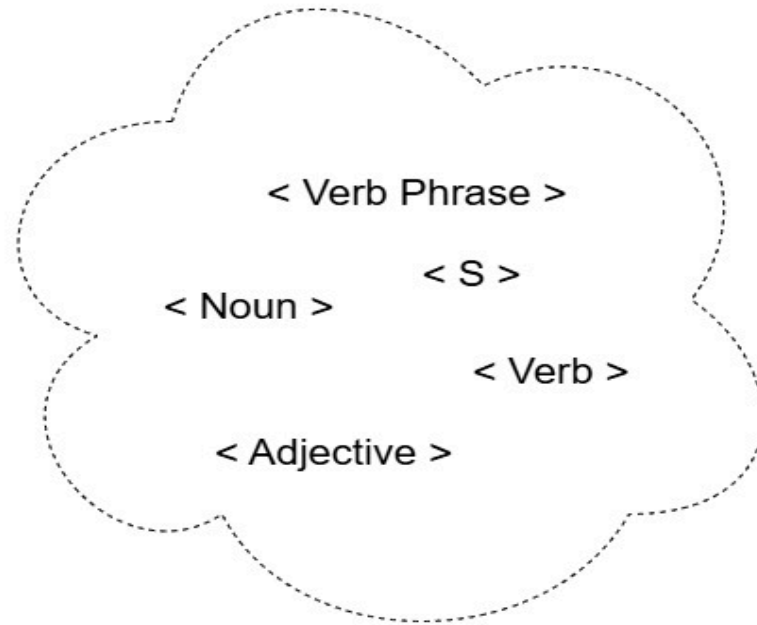


Parse Tree

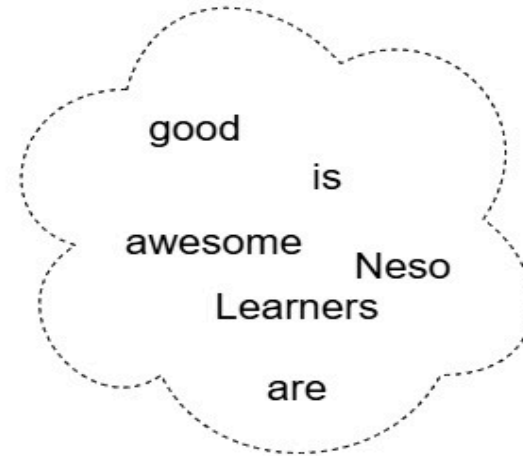
Learners are awesome



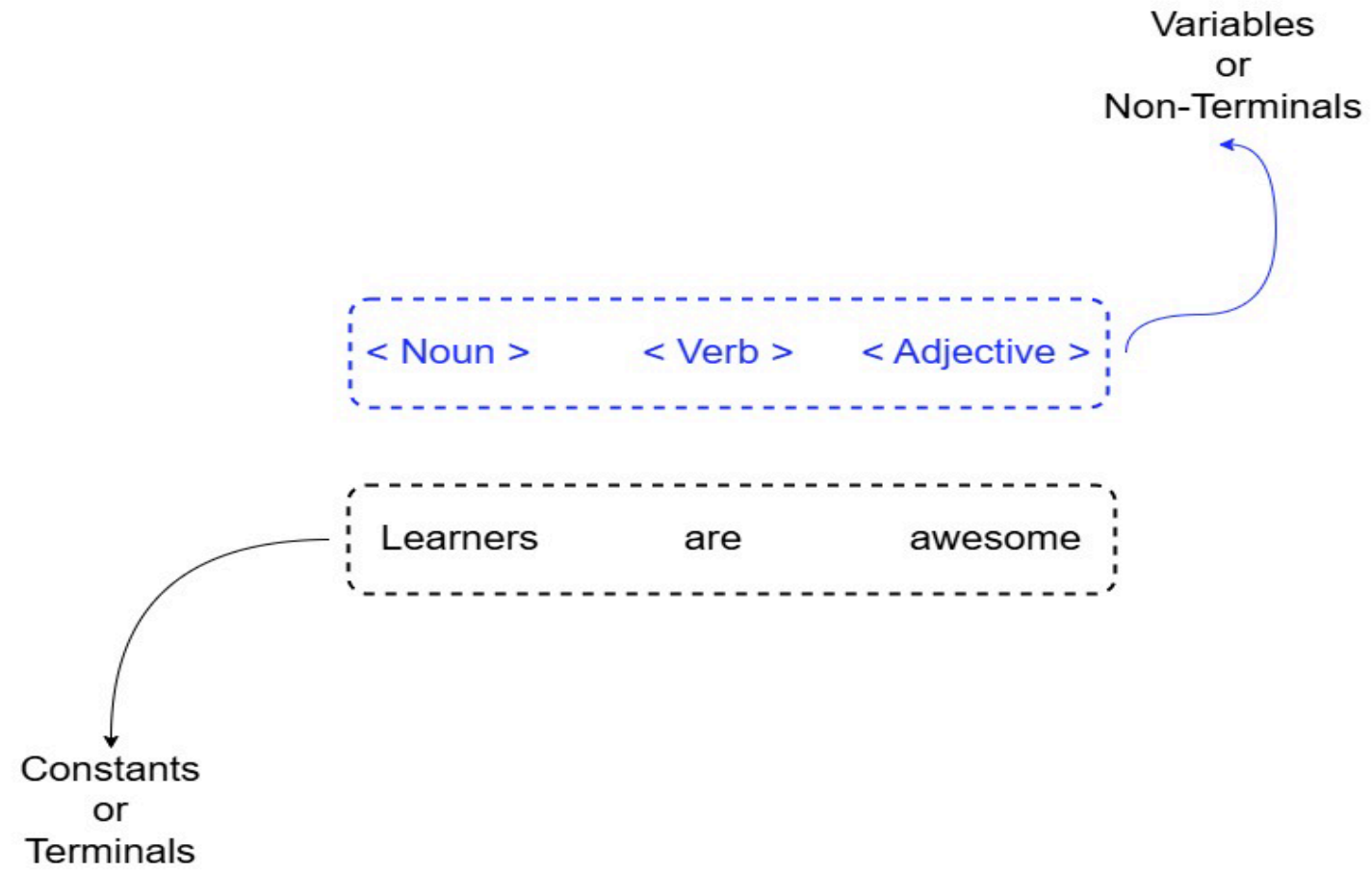


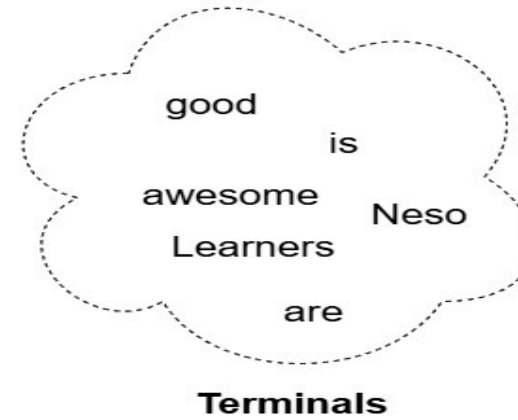
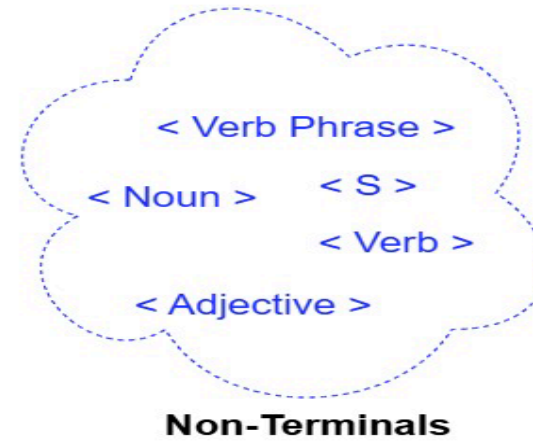
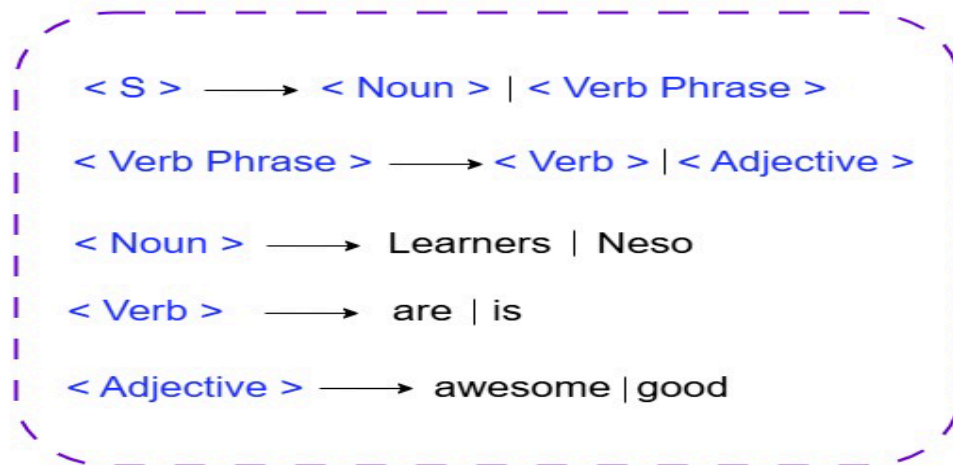


Variables
or
Non-Terminals



Constants
or
Terminals





Example

For example, consider the following grammar:

➤ $N = \{S, A\}$ (where S is the start symbol),

➤ $\Sigma = \{a, b\}$ (alphabet),

➤ P includes the rules:

- $S \rightarrow aSb$
- $S \rightarrow ba$

Example: Generating Strings

Using the above grammar, we can generate strings starting from S:

- Start with S.
- Apply rule 1: $S \rightarrow aSb$.
- Apply rule 1 again: $S \rightarrow aSb$ resulting in $aaSbb$.
- Apply rule 2: $S \rightarrow ba$ resulting in $aababb$.
- We can write this series of choices more briefly, using symbols: $S \rightarrow aSb \rightarrow aaSbb \rightarrow aababb$
- The language of the grammar is then the infinite set $\{a^n bab^n \mid n \geq 0\} = \{ba, abab, aababb, aaababbb, \dots\}$

➤ The grammar generates strings where:

1. The number of a's before the central ba equals the number of b's after it.
2. The central ba is fixed, acting as a "divider."

Examples:

- 0 ba - No a's or b's added.
- 1 abab - One a left, one b right.
- 2 aababb - Two a's left, two b's right.
- 3 aaababbb - Three a's left, three b's right.

The grammar generation

- The number of a's before the central ba equals the number of b's after it.
- The central ba is fixed, acting as a "divider."

nn	String	Explanation
0	ba	No a's or b's added.
1	abab	One a left, one b right.
2	aababb	Two a's left, two b's right.
3	aaababbb	Three a's left, three b's right.